

React

Prepared by: Aamir Pinger



fb.com/AamirPingerOfficial



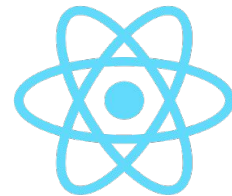
linkedin.com/in/AamirPinger



github.com/AamirPinger

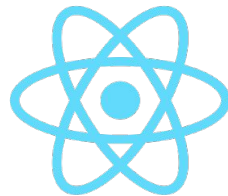
React Book

<https://softchris.github.io/books/react/>



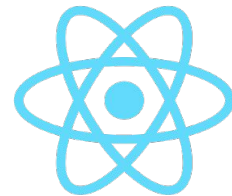
React

- The initial React release was 2013 by Facebook.
- React is a library made over javascript
- In recent years single page applications (SPA) have become popular.
- React is not an SPA framework but a “view” library.
- It is the V in the MVC (Model-View-Controller architectural pattern).



React

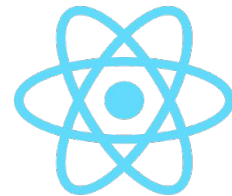
- Model-View-Controller (MVC) is an application model comprised of three dependent layers.
 - The model (data)
 - The view (user interface)
 - The controller (processes that handle input).
- React only enables you to render components as viewable elements in a browser.



React

- A Single Page Application (SPA) differs from a normal web application
- In SPA that you remain on the same URL and thereby the same page, hence the name.
- Traditionally in HTML we create multiple HTML files for multiple page website
- In SPA we create one HTML page and create Routes on them to show different pages

Why React



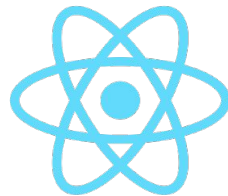
Why React

Because of :

- Its compositional model
- Its declarative rather than imperative
- Unidirectional Data flow/binding
- React is simply javascript

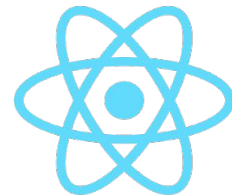
Compositional Model





Compositional model

- Composition is an act or mechanism to combine simple functions to build more complicated ones.
- Why Composition?
- Two things to remember:
 - Simple functions
 - Combination of simple functions to make another function

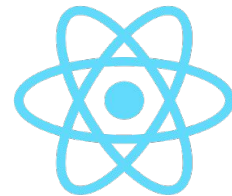


Compositional model

- Let's look at an a simple JS function example:

```
function getTwitterProfile (username) {  
  return 'https://twitter.com/' + username  
}
```

It's a very simple function

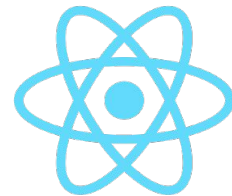


Compositional model

- Similarly, the simple `getTwitterUserPic` function to return url for user twitter profile picture:

```
function getTwitterUserDP (username) {  
  return 'https://avatars.io/twitter/' + username + '/medium'  
}
```

Another very simple function



Compositional model

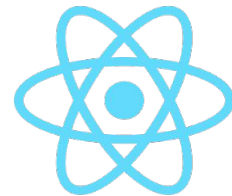
- Let's combine both functions

```
function getTwitterProfileData (username) {  
  return {  
    twitterProfile: getTwitterProfile(username),  
    profilePic: getTwitterUserDP(username)  
  }  
}
```

That is composition!

QUESTION ARISES?

Why three function instead of one directly?



Compositional model

What we did

```
function getTwitterProfile (username) {  
  return 'https://twitter.com ' + username  
}
```

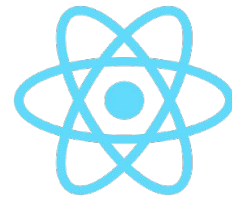
```
function getTwitterUserDP (username) {  
  return 'https://avatars.io/twitter/' + username + '/medium'  
}
```

```
function getTwitterProfileData (username) {  
  return {  
    twitterProfile: getTwitterProfile(username),  
    profilePic: getTwitterUserDP(username)  
  }  
}
```

Function 1

Function 2

Composite
Function



Compositional model

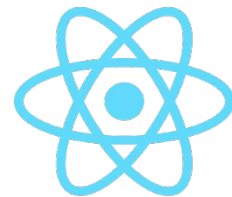
How about

```
function getTwitterProfileData (username) {  
  return {  
    twitterProfile: 'https://twitter.com ' + username,  
  
    profilePic: 'https://avatars.io/twitter/' + username + '/medium'  
  }  
}
```

Non
composite
way

```
function getTwitterProfileData (username) {  
  return {  
    twitterProfile: getTwitterProfile(username),  
    profilePic: getTwitterUserDP(username)  
  }  
}
```

Composite
Function



Compositional model

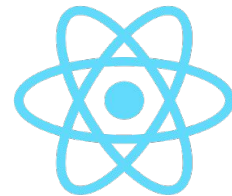
How about

```
function getTwitterProfileData (username) {  
  return {  
    twitterProfile: 'https://twitter.com ' + username,  
  
    profilePic: 'https://avatars.io/twitter/' + username + '/medium'  
  }  
}
```



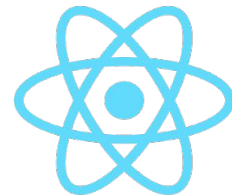
Non
composite
way

- Two separate functions with one composite function is better as it increase reusability.
- There is always a one rule for a good function, a rule of "**DOT**", *Do one thing!*



React & Composition

- In React we rely on composition, heavily!
- In React we create Components to build different sections of a website
- Components are building blocks in React.
- For example following are three different components:
 - `<LandingPage />`
 - `<AboutUs />`
 - `<ContactUs />`
- Currently they are independent component



React & Composition

- In React, we can have a composite component as simple as

```
<LandingPage>
```

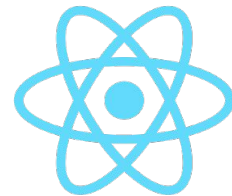
```
  <AboutUs />
```

```
  <ContactUs />
```

```
</LandingPage>
```

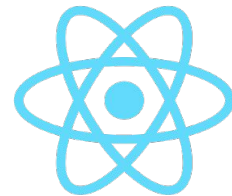
- `<LandingPage>` become parent component and other become child component
- This way we can use individual component blocks to build a big website.

Declarative Nature



Declarative nature

- Most of JavaScript is imperative code.
- We spoon feed each and every step to make javascript aware of how to get desired result.
- Let's take a example water tank level
 - Manual - (Imperative)
 - Auto - (Declarative)



Declarative nature

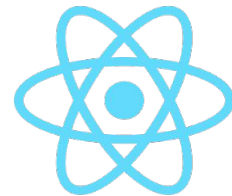
Imperative code!

```
const teachers = ['Zia', 'Irfan', 'Muneeb', 'Aamir']
const titles = []

for (let i = 0; i < teachers.length; i++) {
  titles[i] = 'Mr. ' + teachers[i]
}

console.log(titles)
```

RESULT: ['Mr. Zia', 'Mr. Irfan', 'Mr. Muneeb', 'Mr. Aamir']

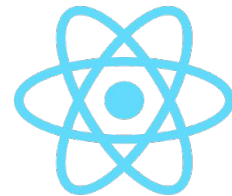


Declarative nature

Declarative code!

```
const teachers = ['Zia', 'Irfan', 'Muneeb', 'Aamir']  
  
const titles = teachers.map(name => `Mr. ` + name )  
  
console.log(titles)
```

RESULT: ['Mr. Zia', 'Mr. Irfan', 'Mr. Muneeb', 'Mr. Aamir']



React - Declarative Nature

In React we will soon going to see declarative code like following

```
<PrintOnBrowser name='Aamir' />
```

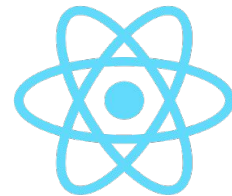
```
<PrintOnBrowser name='Aamir Pinger' />
```

RESULT on Browser:

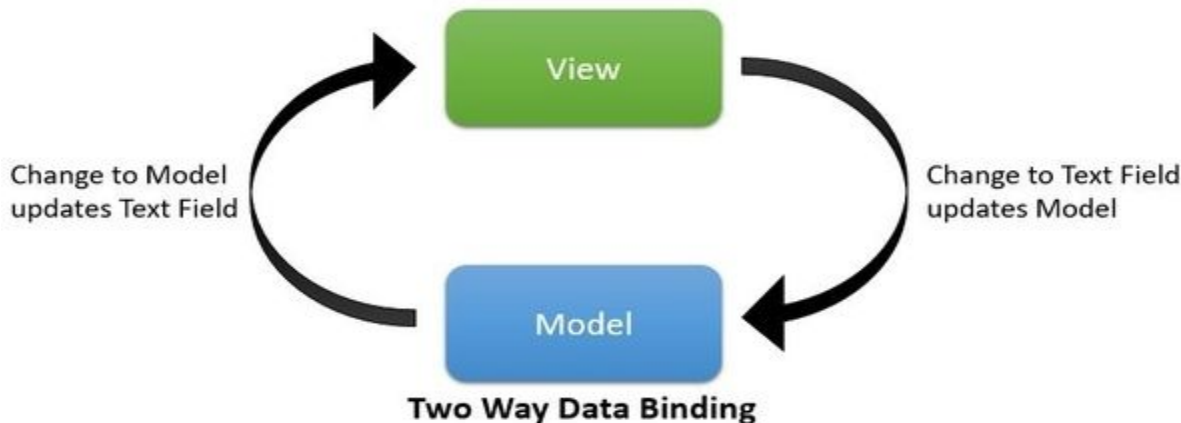
Aamir

Aamir Pinger

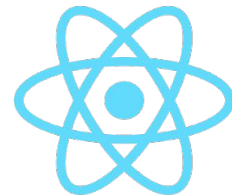
Unidirectional Data Binding



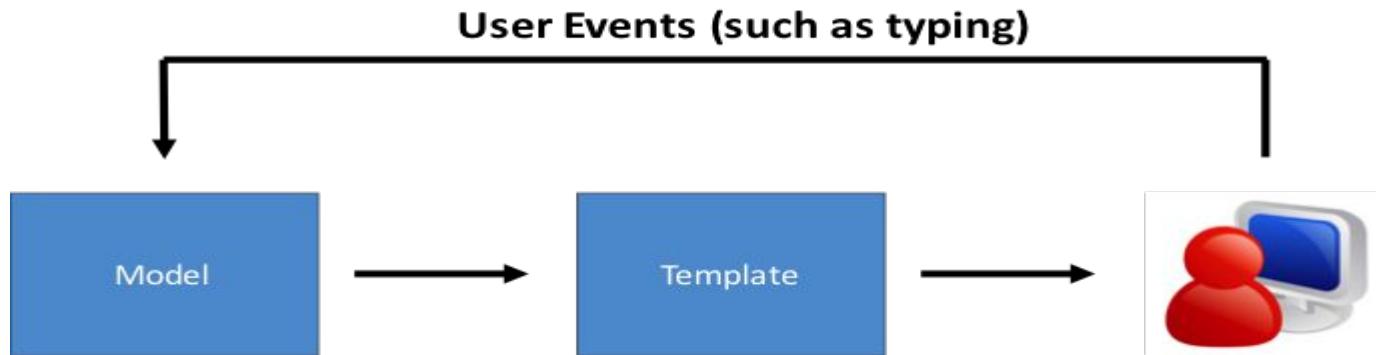
Two way data binding



- Many front-end framework like Angular uses two way data binding.
- Two-way data binding look really great, but when application grows it is hard to determine where the data is actually being updated.



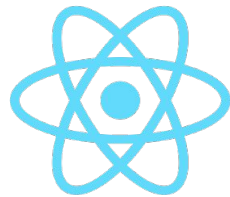
Unidirectional Data Binding



- One-way data binding only propagates changes from the model to the UI, not vice versa.
- It is known as **“single source of truth”**.

React is simply Javascript

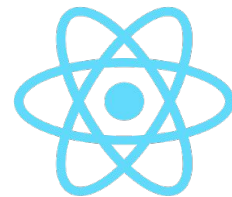




React is simply Javascript

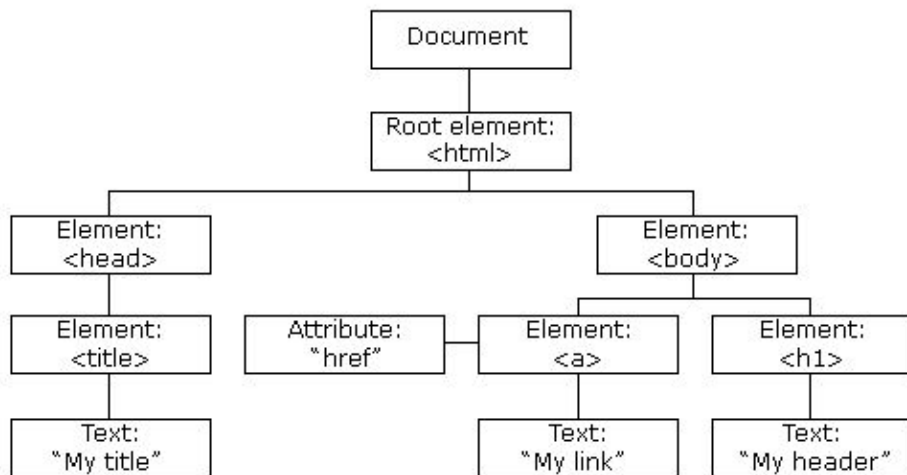
- Uptil now we haven't seen any code other than javascript code
- React is small library based on Javascript
- Even components in React are JavaScript class or function
- Arrow functions, `.map()` and `.filter()` will be seen used extensively in any React code

DOM vs Virtual DOM

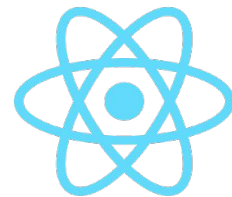


DOM

- DOM is Document Object Model
- It's a programming interface for HTML and XML documents
- When a web page is loaded, the browser creates a Document Object Model of the page.

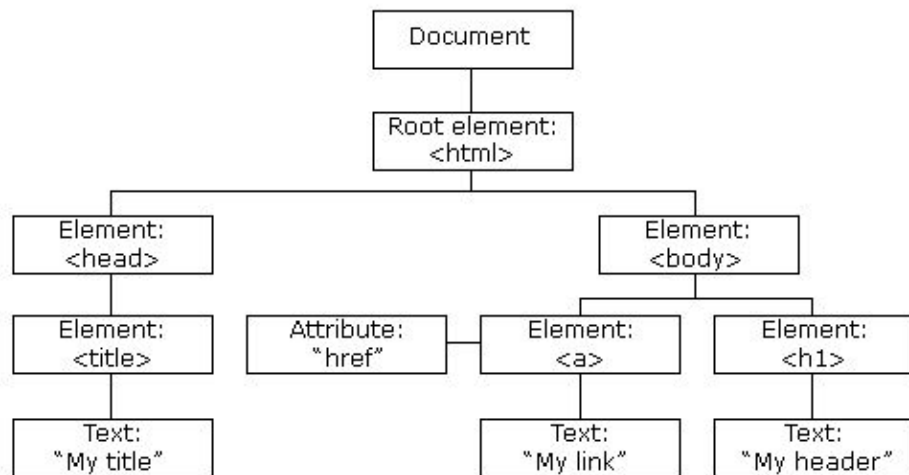


DOM Tree

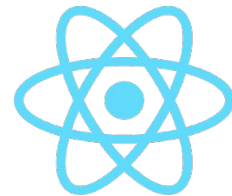


DOM

- The DOM is an object-oriented representation of the web page.
- Scripting language such as JavaScript can modify the document structure, style, and content.



DOM Tree

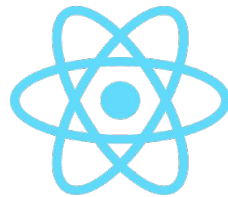


Virtual DOM

- React introduced Virtual DOM (VDOM)
- The VDOM is a programming concept where a virtual representation of a UI is kept in memory
- It's is a tree based on JavaScript objects created with React that resembles a DOM tree
- A process called Reconciliation is used to sync Real DOM with VDOM
- React uses ReactDOM Library updates VDOM and render it on actual DOM

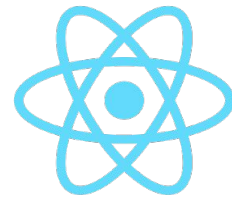
QUESTION ARISES?

Why Virtual DOM is needed?



Why Virtual DOM is needed?

- Making changes in memory (VDOM) is quite faster than updating a complete browser screen (Real DOM)
- React creates first VDOM when application launches and then put everything on browser

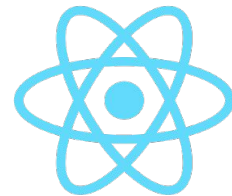


Why Virtual DOM is needed?

- Once app need to update browser screen, React creates new updated VDOM
- Through reconciliation process React find out difference between new and old VDOM
- Lastly only updates the difference to browser (Real DOM)

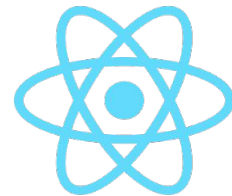
JSX





JSX

- JSX stands for JavaScript XML.
- Browser uses HTML tags to render the content of the webpage
- .html files are used to write HTML tags
- React is javascript and it uses .js files.
- JSX allows us to write HTML tags in .js files.
- JSX makes it easier to write and add HTML in React.



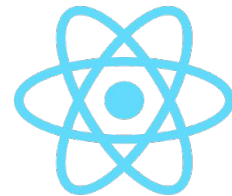
JSX Example

- In HTML we write

```
<div> Hello world </div>
```

- In React

```
React.createElement('div', null, 'Hello World')
```



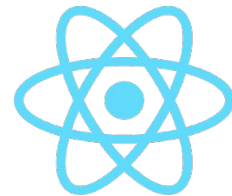
JSX Example

- In HTML we write

```
<div>  
  <h1> Hello world </h1>  
</div>
```

- In React

```
React.createElement(  
  'div',  
  null,  
  React.createElement(h1, null, 'Hello World')  
)
```



JSX

- For nested tags React gives us easy way to write, for example

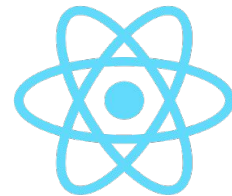
In HTML

```
<div>
  <h1>Some title</h1>
  <div>Some content</div>
</div>
```

In React

```
var myElement = (
  <div>
    <h1>Some title</h1>
    <div>Some content</div>
  </div>
)

ReactDOM.render(<myElement />,
  document.getElementById('root')
)
```

JSX

- Important thing to remember, JSX needs to have one parent.

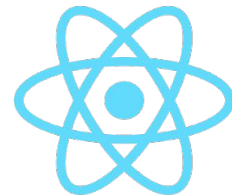
This will throw error



```
var myElement = (  
  <h1>Some title</h1>  
  <div>Some content</div>  
)  
  
ReactDOM.render(<myElement />,  
  document.getElementById('root')  
)
```



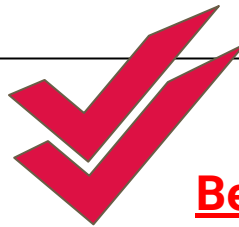
```
var myElement = (  
  <div>  
    <h1>Some title</h1>  
    <div>Some content</div>  
  </div>  
)  
  
ReactDOM.render(<myElement />,  
  document.getElementById('root')  
)
```



JSX

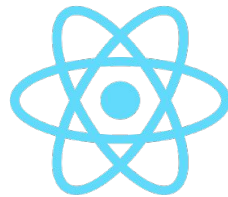
- We can also use `React.Fragment` instead of `<div>`

```
var myElement = (  
  <React.Fragment>  
    <h1>Some title</h1>  
    <div>Some content</div>  
  </React.Fragment>  
)  
ReactDOM.render(<myElement />, document.getElementById('root'))
```



Better way

Setting up React

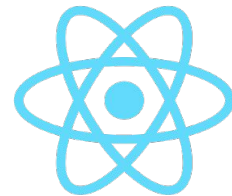


Setting up React

- Easiest way to setup React environment is by using **create-react-app**
- **Create-react-app** scaffold a basic React application with ease and will get you up and running in no time.

To install create-react-app

```
aamir@ap-linux:~$ npm install create-react-app -g
```



Setting up React

To scaffold React application

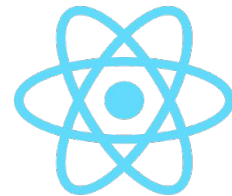
```
aamir@ap-linux:~$ npx create-react-app my-app
```

```
aamir@ap-linux:~$ cd my-app
```

```
aamir@ap-linux:~/my-app$ npm start
```

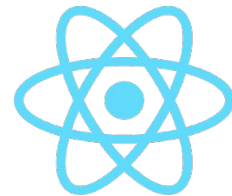
<http://localhost:3000>

Our first React component



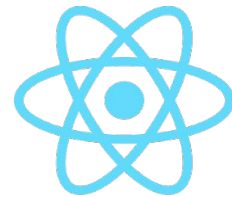
Our first React component

- There are few types of component from which we will be working with following two types in this course
 - Class based Component
 - Function based Component



Class based Component

```
class MyComponent extends React.Component {  
  render() {  
    return (  
      <div>  
        <h1> Hello world </h1>  
      </div>  
    )  
  }  
}  
  
ReactDOM.render(  
  <MyComponent />,  
  document.getElementById('root')  
)
```

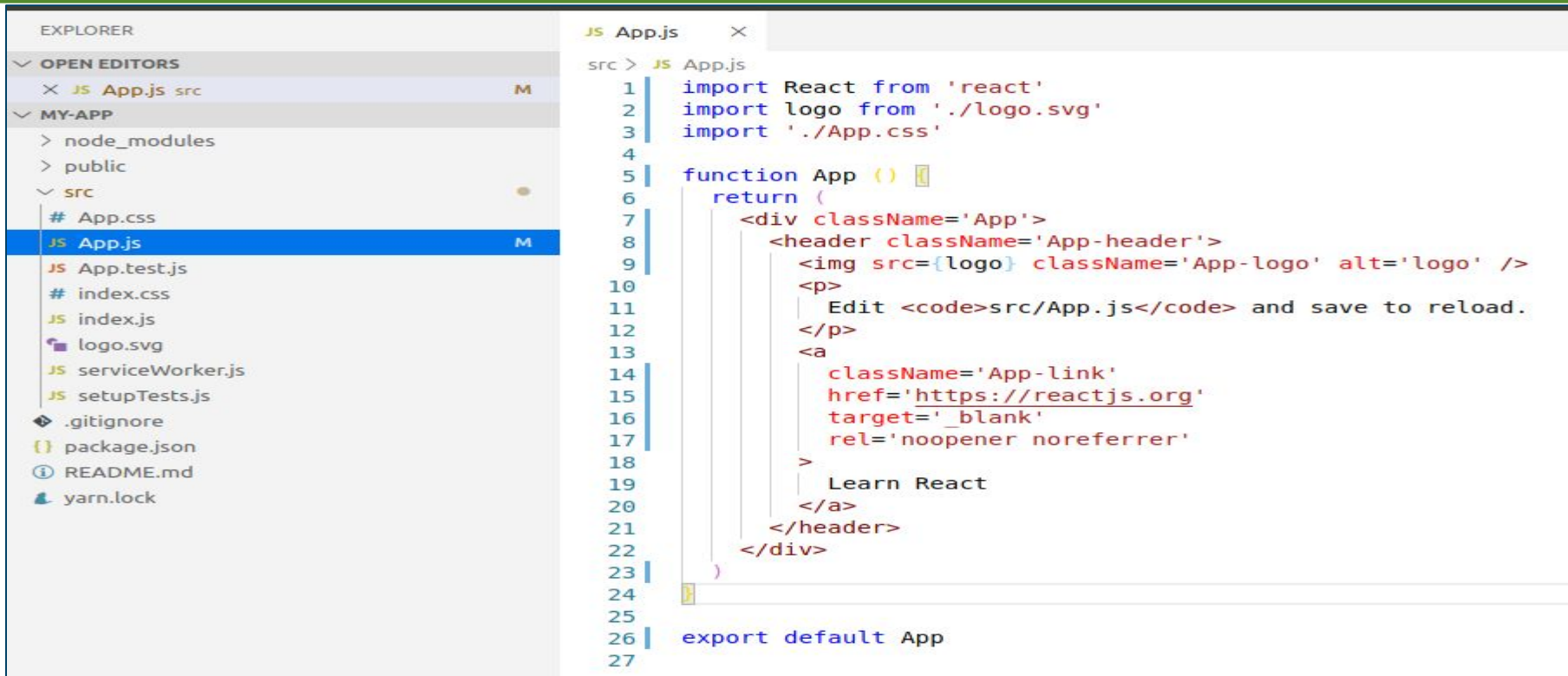
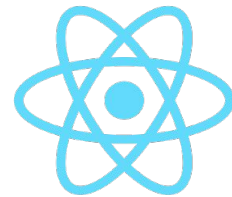



Functional based Component

```
function MyComponent () {  
  return (  
    <div>  
      <h1> Hello world </h1>  
    </div>  
  )  
}  
  
ReactDOM.render(  
  <MyComponent />,  
  document.getElementById('root')  
)
```

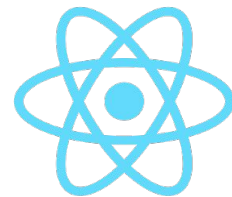
Create-react-app generated
files

Directory and files after create-react-app



The screenshot displays the Visual Studio Code interface. On the left, the Explorer sidebar shows the project structure under 'MY-APP'. The 'src' directory is expanded, listing files like 'App.css', 'App.js', 'App.test.js', 'index.css', 'index.js', 'logo.svg', 'serviceWorker.js', and 'setupTests.js'. 'App.js' is selected. The main editor area shows the content of 'App.js', which includes imports for React, a logo SVG, and App.css, followed by a function App() that returns a JSX element. The JSX element contains a header with a logo and a link to 'https://reactjs.org'. The file is exported as the default.

```
src > JS App.js
1  import React from 'react'
2  import logo from './logo.svg'
3  import './App.css'
4
5  function App () {
6    return (
7      <div className='App'>
8        <header className='App-header'>
9          <img src={logo} className='App-logo' alt='logo' />
10         <p>
11           Edit <code>src/App.js</code> and save to reload.
12         </p>
13         <a
14           className='App-link'
15           href='https://reactjs.org'
16           target='_blank'
17           rel='noopener noreferrer'
18         >
19           Learn React
20         </a>
21       </header>
22     </div>
23   )
24
25
26   export default App
27
```

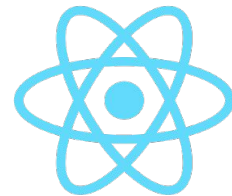


Our first React component

A screenshot of a code editor interface. On the left is the 'EXPLORER' sidebar showing a file tree. The tree has a 'MY-APP' folder containing 'node_modules', 'public', and 'src'. The 'src' folder is expanded, showing files like 'App.css', 'App.js' (selected), 'App.test.js', 'index.css', 'index.js', 'logo.svg', 'serviceWorker.js', and 'setupTests.js'. Below these are '.gitignore', 'package.json', 'README.md', and 'yarn.lock'. The main editor area shows the content of 'App.js'. It starts with three import statements: 'import React from \'react\'', 'import logo from \'../logo.svg\'', and 'import \'../App.css\''. Then, a class 'MyComponent' is defined, extending 'React.Component'. The 'render' method returns a JSX element: a 'div' containing an 'h1' with the text 'Hello world'. Finally, a function 'App' is defined that returns a 'MyComponent' element, and 'App' is exported as the default export.

```
src > JS App.js
1 | import React from 'react'
2 | import logo from '../logo.svg'
3 | import '../App.css'
4 |
5 | class MyComponent extends React.Component {
6 |   render () {
7 |     return (
8 |       <div>
9 |         <h1> Hello world </h1>
10 |      </div>
11 |    )
12 |   }
13 | }
14 |
15 | function App () {
16 |   return (
17 |     <MyComponent />
18 |   )
19 | }
20 |
21 | export default App
```

<https://github.com/aamirpinger/react-fundamental-slides-code/blob/master/myFirstComponent.js>



Component with .map method

```
import React from 'react'

class MyComponent extends React.Component {
  render () {
    const cityArray = ['Karachi', 'Lahore', 'Peshawar', 'Quetta']

    return (
      <ul>
        {
          cityArray.map(city => <li key={city}> {city} </li>)
        }
      </ul>
    )
  }
}

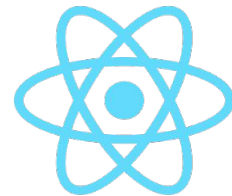
function App () {
  return (<MyComponent />)
}

export default App
```

ComponentWithDotMap.js

Result

- Karachi
- Lahore
- Peshawar
- Quetta



Component with .filter method

ComponentWithDotFilter.js

```
import React from 'react'

class MyComponent extends React.Component {
  render () {
    const cityArray = ['Karachi', 'Lahore', 'Peshawar', 'Quetta']

    const shortListedCities = cityArray.filter(city => city.length > 6)
    return (
      <ul>
        {
          shortListedCities.map(city => <li key={city}> {city} </li>)
        }
      </ul>
    )
  }
}

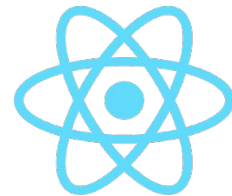
function App () {
  return (<MyComponent />)
}

export default App
```

Result

- Karachi
- Peshawar

Component Reusability



Component Reusability

```
import React from 'react'

class MyComponent extends React.Component {
  render () {
    const cityArray = ['Karachi', 'Lahore', 'Peshawar', 'Quetta']

    const shortListedCities = cityArray.filter(city => city.length > 6)
    return (
      <ul>
        { shortListedCities.map(city => <li key={city}> {city} </li>)}
      </ul>
    )
  }
}

function App () {
  return (
    <div>
      <MyComponent />
      <MyComponent />
    </div>
  )
}

export default App
```

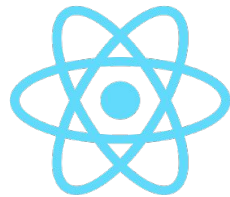
ComponentWithDotFilter.js

Result

- Karachi
- Peshawar
- Karachi
- Peshawar

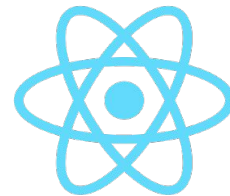
props





props

- React allow us to send data or functions from parent component to it's child component
- These data or functions are called props
- Adding props, or properties to your component means we add attributes to our component element.



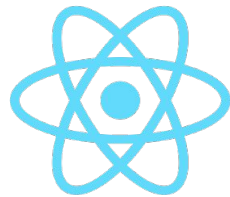
props

Component call WITHOUT props

```
<myElement />
```

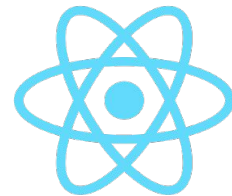
Component call WITH props

```
<myElement  
  name='Aamir'  
  myfunc={() => console.log('hello world')}  
/>
```



props

- This means we can pass data from an outer component to an inner component.
- That data can be either data that we want to render or a function that we want to invoke.



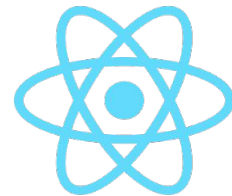
props

PassingProps.js

```
src > JS App.js
1  import React from 'react'
2
3  class MyComponent extends React.Component {
4    render () {
5      const cityArray = this.props.cityArray
6
7      const shortListedCities = cityArray.filter(city => city.length > 6)
8      return (
9        <ul>
10         {shortListedCities.map(city => <li key={city}> {city} </li>)}
11       </ul>
12     )
13   }
14 }
15
16 function App () {
17   const cityArray1 = ['Karachi', 'Lahore', 'Peshawar', 'Quetta']
18   const cityArray2 = ['Hyderabad', 'Islamabad', 'Sawat', 'Gawader']
19
20   return (
21     <div>
22       <MyComponent cityArray={cityArray1} />
23       <MyComponent cityArray={cityArray2} />
24     </div>
25   )
26 }
27 export default App
```

Result

- Karachi
- Peshawar
- Hyderabad
- Islamabad
- Gawader



props

PassingPropsAndFunction.js

```
1  import React from 'react'
2
3  class MyComponent extends React.Component {
4    render () {
5      const cityArray = this.props.cityArray
6
7      const shortListedCities = cityArray.filter(city => city.length > 6)
8
9      this.props.myFunction()
10
11     return (
12       <ul>
13         {shortListedCities.map(city => <li key={city}> {city} </li>)}
14       </ul>
15     )
16   }
17 }
18
19 function App () {
20   const cityArray1 = ['Karachi', 'Lahore', 'Peshawar', 'Quetta']
21   const cityArray2 = ['Hyderabad', 'Islamabad', 'Sawat', 'Gawader']
22
23   const myFunction = () => alert('Hello World')
24
25   return (
26     <div>
27       <MyComponent cityArray={cityArray1} myFunction={myFunction} />
28       <MyComponent cityArray={cityArray2} myFunction={myFunction} />
29     </div>
30   )
31 }
32
33 export default App
```

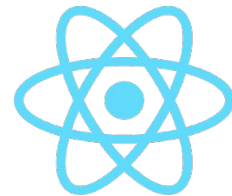
Result

localhost:3000

localhost:3000 says
Hello World

OK

Passing Props to Functional component



Props to Functional Component

PropsToFunctionalComp.js

```
1 import React from 'react'
2
3 class MyComponent extends React.Component {
4   render () {
5     const cityArray = this.props.cityArray
6
7     const shortListedCities = cityArray.filter(city => city.length > 6)
8
9     // this.props.myFunction()
10
11     return (
12       <ul>
13         {shortListedCities.map(city => <li key={city}> {city} </li>)}
14       </ul>
15     )
16   }
17 }
18
19 function MyFunctionalComponent (props) {
20   return <h1>{props.heading}</h1>
21 }
22
23 function App () {
24   const cityArray1 = ['Karachi', 'Lahore', 'Peshawar', 'Quetta']
25   const cityArray2 = ['Hyderabad', 'Islamabad', 'Sawat', 'Gawader']
26
27   const myFunction = () => alert('Hello World')
28
29   return (
30     <div>
31       <MyFunctionalComponent heading='Cities List' />
32       <MyComponent cityArray={cityArray1} myFunction={myFunction} />
33       <MyComponent cityArray={cityArray2} myFunction={myFunction} />
34     </div>
35   )
36 }
37
38 export default App
```

Result

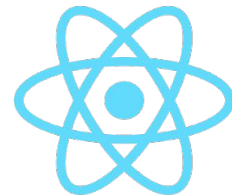
← → ↺ ⓘ localhost:3000

Cities List

- Karachi
- Peshawar
- Hyderabad
- Islamabad
- Gawader

PropTypes



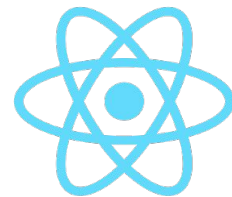


PropTypes

- React components are meant to be reuse
- In a bigger application soon teams will work on a project and reuse other's components
- Props is integral part of component

QUESTION ARISES?

What if wrong props are passed to our component
e.g. array instead of object



Wrong props

```
1 import React from 'react'
2
3 class MyComponent extends React.Component {
4   render () {
5     const cityArray = this.props.cityArray
6
7     const shortListedCities = cityArray.filter(city => city.length > 6)
8
9     this.props.myFunction()
10
11    return (
12      <ul>
13        {shortListedCities.map(city => <li key={city}> {city} </li>)}
14      </ul>
15    )
16  }
17 }
18
19 function App () {
20   const cityArray1 = ['Karachi', 'Lahore', 'Peshawar', 'Quetta']
21   const cityArray2 = ['Hyderabad', 'Islamabad', 'Sawat', 'Gawader']
22
23   const cityObj = { city1: 'Karachi', city2: 'Lahore', city3: 'Peshawar', city4: 'Quetta' }
24
25   const myFunction = () => alert('Hello World')
26
27   return (
28     <div>
29       <MyComponent cityArray={cityObj} myFunction={myFunction} />
30       <MyComponent cityArray={cityArray2} myFunction={myFunction} />
31     </div>
32   )
33 }
34
35 export default App
```

Result

localhost:3000

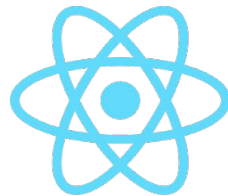
TypeError: cityArray.filter is not a function

MyComponent.render
src/App.js:7

```
4 | render () {
5 |   const cityArray = this.props.cityArray
6 |
> 7 |   const shortListedCities = cityArray.filter(city => city.length > 6)
    |                                     ^
8 |   this.props.myFunction()
9 |
10 | }
```

View compiled

► 16 stack frames were collapsed.



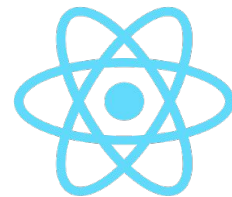
PropTypes

- By using **PropTypes** library, we can define the data type we expect if someone uses our component and passes props to it.
- It will warn us during development in the console of browser if unexpected data type passed

To install PropTypes

```
aamir@ap-linux:~$ npm install prop-types
```

More details: <https://github.com/facebook/prop-types>

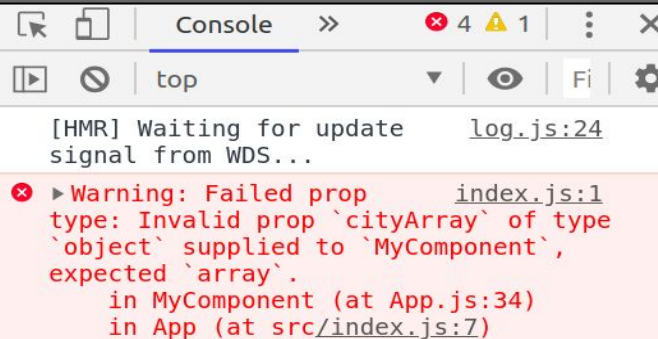


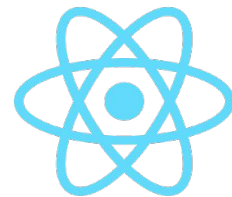
PropTypes

PropTypesExample.js

```
1 import React from 'react'
2 import PropTypes from 'prop-types'
3
4 class MyComponent extends React.Component {
5   render () {
6     const cityArray = this.props.cityArray
7
8     const shortListedCities = cityArray.filter(city => city.length > 6)
9
10    // this.props.myFunction()
11
12    return (
13      <ul>
14        {shortListedCities.map(city => <li key={city}> {city} </li>)}
15      </ul>
16    )
17  }
18 }
19
20 MyComponent.propTypes = {
21   cityArray: PropTypes.array.isRequired
22 }
23
24 function App () {
25   const cityArray1 = ['Karachi', 'Lahore', 'Peshawar', 'Quetta']
26   const cityArray2 = ['Hyderabad', 'Islamabad', 'Sawat', 'Gawader']
27
28   const cityObj = { city1: 'Karachi', city2: 'Lahore', city3: 'Peshawar', city4: 'Quetta' }
29
30   const myFunction = () => alert('Hello World')
31
32   return (
33     <div>
34       <MyComponent cityArray={cityObj} myFunction={myFunction} />
35       <MyComponent cityArray={cityArray2} myFunction={myFunction} />
36     </div>
37   )
38 }
39
40 export default App
```

Result





PropTypes

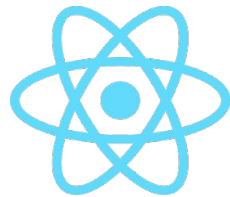
- It is considered a best practice to always use this library
- It is also recommended to make it part of the class

PropTypeInClass.js

```
1 import React from 'react'
2 import PropTypes from 'prop-types'
3
4 class MyComponent extends React.Component {
5   static propTypes = {
6     cityArray: PropTypes.array.isRequired
7   }
8
9   render () {
10     const cityArray = this.props.cityArray
11
12     const shortListedCities = cityArray.filter(city => city.length > 6)
13
14     // this.props.myFunction()
15
16     return (
17       <ul>
18         {shortListedCities.map(city => <li key={city}> {city} </li>)}
19       </ul>
20     )
21   }
22 }
```

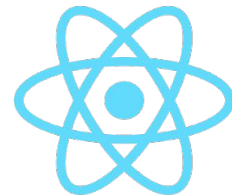
State





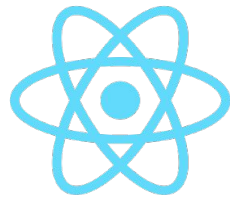
State

- The heart of every React component is its “state”
- It is simply an Javascript object that determines how that component renders & behaves
- In other words, “state” is what allows you to create components that are dynamic and interactive



State

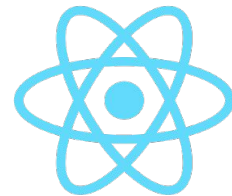
- Props from parent component are immutable
- State is mutable
- Change in state re-renders your component at browser
- Every change in state provide updated data to the user



State

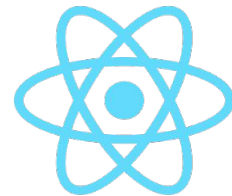
- By rules State can only be declared for class components
- That's what makes you decide whether to use class component or functional component
- By using React Hooks you can now have React state in the functional component

Place to declare State



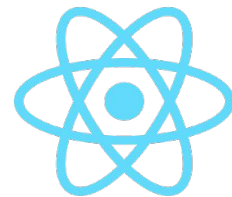
Where state should be declared

- State can only be mutate by owner component
- You can make one parent component with state and pass the state value to its child components as a props
- Props cannot change they are immutable
- If child component need to make any changes in state it can request its parent (owner) component to make changes



Where state should be declared

- Parent component has to pass a function as a props which will be used by child component to request a state change
- Once changes are done by parent, react automatically pass updated state value to all the child component where parent component have passed state values as a props
- This way React insures single source of truth and unidirectional data model



State

StateExample.js

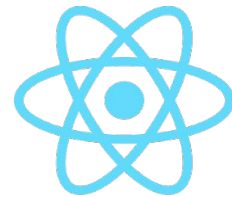
```
c > JS App.js
1  import React from 'react'
2  import PropTypes from 'prop-types'
3
4  class MyComponent extends React.Component {
5    state = {
6      cityArray: ['Karachi', 'Lahore', 'Peshawar', 'Quetta']
7    }
8
9    render () {
10     return (
11       <ul>
12         {
13           this.state.cityArray.map(city => <li key={city}> {city} </li>)
14         }
15       </ul>
16     )
17   }
18 }
19
20 function App () {
21   return (
22     <div>
23       <MyComponent />
24     </div>
25   )
26 }
27
28 export default App
```

Result

← → ↻ ⓘ localhost:3000

- Karachi
- Lahore
- Peshawar
- Quetta

Deconstructing



Destructuring - Array

- The destructuring assignment syntax is a JavaScript expression that makes it possible to unpack values from arrays, or properties from objects, into distinct variables

```
let myArray = ["Aamir", "Pinger", 2]
```

```
let [firstName, lastName, degrees] = myArray
```

```
console.log(firstName)
```

```
console.log(lastName)
```

```
console.log(degrees)
```

```
let sentence = `${firstName} ${lastName} has ${degrees} Masters degrees.`
```

```
console.log(sentence)
```

RESULT

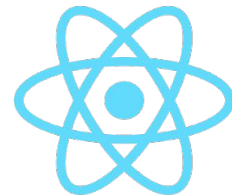
Aamir

Pinger

2

Aamir Pinger has 2 Masters degrees.

[DestructuringArray1.js](#)



Destructuring - Array

```
let myArray = ["Aamir", "Pinger", 2]
```

```
let [firstName, lastName] = myArray
```

```
console.log(firstName)
```

```
console.log(lastName)
```

```
console.log(degrees)
```

```
let sentence = `${firstName} ${lastName} has ${degrees} Masters degrees.`
```

```
console.log(sentence)
```

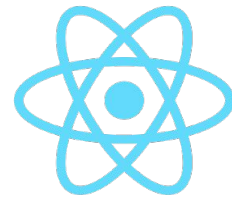
RESULT

Aamir

Pinger

degrees is not defined

[DestructuringArray2.js](#)



Destructuring - Object

```
let myObject = {firstName: "Aamir", lastName: "Pinger", degrees: 2}
```

```
console.log(myObject.firstName)
```

```
console.log(myObject.lastName)
```

```
console.log(myObject.degrees)
```

```
let {firstName, lastName, degrees} = myObject
```

```
console.log(firstName)
```

```
console.log(lastName)
```

```
console.log(degrees)
```

```
let sentence = `${firstName} ${lastName} has ${degrees} Masters degrees`
```

```
console.log(sentence)
```

RESULT

Aamir

Pinger

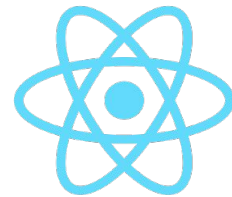
2

Aamir

Pinger

2

Aamir Pinger has 2 Masters degrees.



Destructuring - Object

```
let myObject = {firstName: "Aamir", lastName: "Pinger", degrees: 2}
```

```
console.log(myObject.firstName)
```

```
console.log(myObject.lastName)
```

```
console.log(myObject.degrees)
```

```
let {firstName, lastName} = myObject
```

```
console.log(firstName)
```

```
console.log(lastName)
```

```
console.log(degrees)
```

```
let sentence = `${firstName} ${lastName} has ${degrees} Masters degrees`
```

```
console.log(sentence)
```

RESULT

Aamir

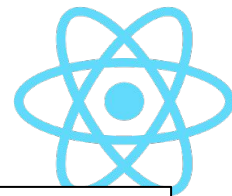
Pinger

2

Aamir

Pinger

degrees is not defined

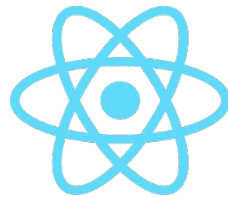


Destructure State and Props

DestructureStateAndProps.js

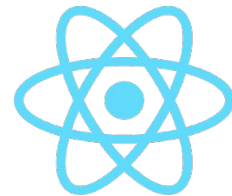
```
> JS App.js
1  import React from 'react'
2
3  class MyComponent extends React.Component {
4    state = {
5      cityArray: ['Karachi', 'Lahore', 'Peshawar', 'Quetta']
6    }
7
8    render () {
9      const { myFunction } = this.props
10     const { cityArray } = this.state
11
12     myFunction()
13
14     return (
15       <ul>
16         {
17           cityArray.map(city => <li key={city}> {city} </li>)
18         }
19       </ul>
20     )
21   }
22 }
23
24 function MyFunctionalComponent (props) {
25   const { heading } = props
26   return <h1>{heading}</h1>
27 }
28
29 function App () {
30   const myFunction = () => alert('Hello World')
31
32   return (
33     <div>
34       <MyFunctionalComponent heading='Cities List' />
35       <MyComponent myFunction={myFunction} />
36     </div>
37   )
38 }
39
40 export default App
```

Changing State



Changing State

- To change state we use **setState** method
- Beauty of setState method is no matter if your state got many key value pairs, you can just pass specific keys and their new values you want to update
- Once setState method updates the state it rerenders that component and your browser gets updated data



setState Method

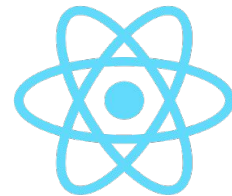
- setState function in React is an asynchronous function
- There are two variations of using setState:
 - Object-based approach
 - Functional approach

```
this.setState(  
  { cityArray: [...this.state.cityArray, value] }  
)
```

OBJECT-BASED APPROACH

```
this.setState(  
  (prevState) => (  
    { cityArray: [...prevState.cityArray, value] }  
  )  
)
```

FUNCTIONAL APPROACH



setState Method

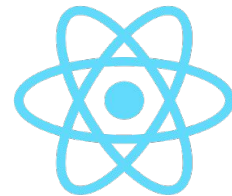
Add city name example Code:

[AddCityInState.js](#)

Remove city name example Code:

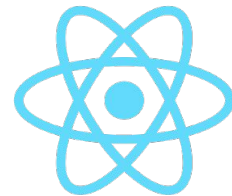
[RemoveCityFromState.js](#)

Forms



Forms

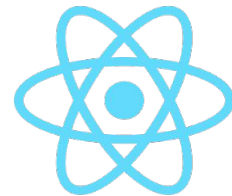
- Form is not new in react, it is same as we used in HTML
- Form is helpful to group multiple input fields and validate on submit
- These input fields can be any of the HTML input field types. Examples are:
 - Textbox
 - TextArea
 - Checkbox
 - Date input
 - Radio
 - Select



Form Example

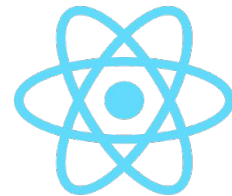
Form example Code:
[FormExample.js](#)

Conditional rendering



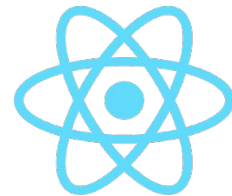
Conditional rendering

- Conditional rendering is about deciding what to render and when
- It is simply rendering something if provided condition is matched
- For example:
 - Maximum limit message component
 - Render an error message instead of alert in case of no city name provided on submit



Conditional rendering

- There are different approaches we can have here:
 - render if true (if else)
 - ternary expression (condition ? true : false)
 - Short circuit evaluation (&&, ||)

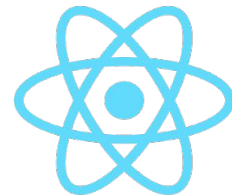


Conditional rendering Example

Conditional rendering example Code:
[ConditionalRendering.js](#)

Styling

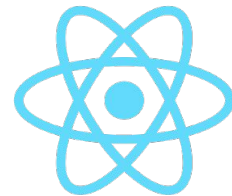




Styling

- Styling is also a important aspect to look when developing a web app
- No matter how efficient you have made you web app, if you did not made a good UI/UX, user may not use it
- React provide us multiple ways to use styling
- Some of the widely used practices are
 - CSS files
 - Inline Styling
 - 3rd party libraries like *Styled Components*

Styling with CSS files

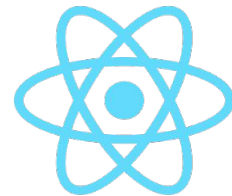


Styling with CSS files

CSS styling example Code:

[CSS-Styling/](#)

Inline Styling



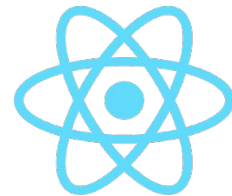
Inline Styling

Inline styling example Code:
[InlineStyling.js](#)

List of all supported HTML attributes that can be passed along to the React element
<https://facebook.github.io/react/docs/dom-elements.html#all-supported-html-attributes>

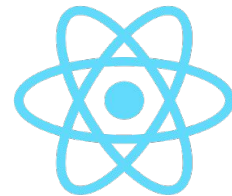
Styled Component





Styled Component

- CSS files have few issues like these are global in scope
- Similar class name could cause side effects to other part of website
- Inline styling though not recommended to use but it is sometimes required to make small dynamic styling
- Styled components is a library that give us solution for above all
- Using style components you don't need css files, you create styled HTML element in JS files and use them wherever you need them



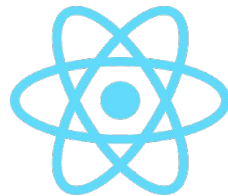
Styled Component

Styled Component example Code:
[CSS-Styling/styled-component-example.js](#)

To install styled-components

```
aamir@ap-linux:~$ npm install styled-components
```

Images in React



Images

- Rendering images in a react is little different
- In HTML you write

```

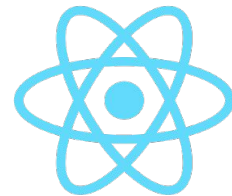
```

- In React you have to import a file right in a JavaScript module
- This tells webpack to include that file in the bundle during the project start.

img tag example Code:

[CSS-Styling/imgTagExample.js](#)

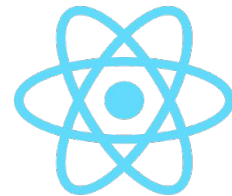
Assignment # 1



Assignment # 1

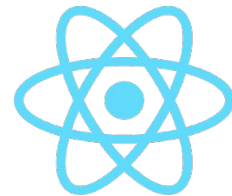
- Create a one page our country profile
- Must be built using multiple components
- Should be having good styling (use any approach for styling or any library you like)
- The page should have a country's map and flag images
- Different component showing different information
- Should have a option to add comments that has to be stored and handled using react state
- Must use destructuring, State, Props, propTypes, Forms, styling, and images.

Lifecycle Events



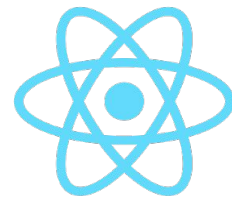
Lifecycle Events

- When developing in React, every Component follows a cycle from
 - When it's created and Mounted on the DOM
 - Got updated during its presence on DOM
 - To when it is unmounted from DOM and destroyed
- This is what we refer to as the Component lifecycle



Lifecycle Events

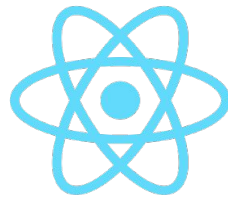
- The React component which extends `React.Component` goes through the following phases:
 - Mounting
 - Updating
 - Unmounting
- React offer us various methods which are invoked at different phases of the lifecycle of a component.



Lifecycle Events

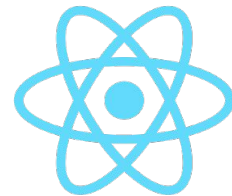
- Mounting phase is initialization of the component when the instance of the component is being created and inserted into the DOM
- Updating phase comes when component is re-rendered due to change in props or state etc
- Unmounting phase is when unmounting or removal of component from the DOM

Why lifecycle events?



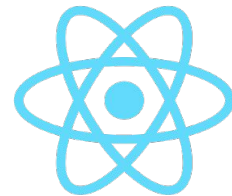
Why lifecycle events?

- Render method should be pure and should not have any side effects
- Pure method means you get the same output every time when provided same input.
- Component's `render()` function should be pure, meaning that it does not modify component state



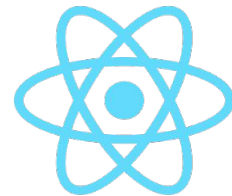
Why lifecycle events?

- `render()` Is meant for Rendering, Only!
- As we said its pure and should return same result when provided same input so that means should not make
 - Any HTTP requests
 - Fetch data that's used to display the content
 - Modify the DOM
 - Or call any function that does the above 3



Lifecycle Events

- There are special methods bound to the every component
- Those methods are triggered on some certain time during component lifecycle
- We can use those methods to do thing which we don't do in render()



Lifecycle Events

- React offer us various methods which are invoked at different phases of the lifecycle of a component.

Mounting

1. constructor()
2. static getDerivedStateFromProps()
3. render()
4. componentDidMount()

Updating

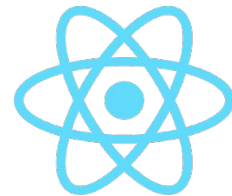
1. static getDerivedStateFromProps()
2. shouldComponentUpdate()
3. render()
4. getSnapshotBeforeUpdate()
5. componentDidUpdate()

Unmounting

1. componentWillUnmount()

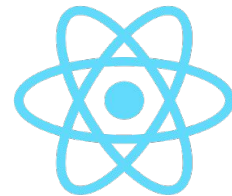
constructor()





constructor()

- Constructors are not react specific or even javascript specific
- They are specific to the inheritance in OOP
- When the component class is created in memory, the constructor is the first method called, so it's the right place to initialize everything – state included
- Earlier we need to define constructor method explicitly but babel now transpile state directly without writing it in constructor()



constructor()

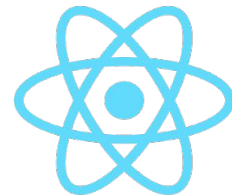
```
class App extends Component {
  constructor (props) {
    super(props)
    this.state = {
      firstName: 'Aamir'
    }

    this.printMessage = this.printMessage.bind(this)
  }

  printMessage () {
    return <h1>`${this.state.firstName} ${this.props.message}`</h1>
  }

  render () {
    return this.printMessage()
  }
}

export default App
```

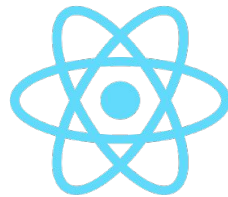


constructor()

```
class App extends Component {  
  state = {  
    firstName: 'Aamir'  
  }  
  
  printMessage () {  
    return <h1>`${this.state.firstName} ${this.props.message}`</h1>  
  }  
  
  render () {  
    return this.printMessage()  
  }  
}  
  
export default App
```

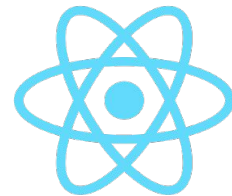
componentDidMount()





componentDidMount()

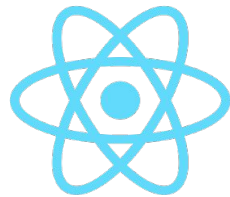
- `componentDidMount()` is invoked only ONCE immediately after a component is mounted (users will have a display on their browsers)
- If you need to load data from a remote endpoint (API Calls), this is a good place
- Initializing state also should be done here
- Setting state in this method will trigger a re-render to update the data everywhere



componentDidMount()

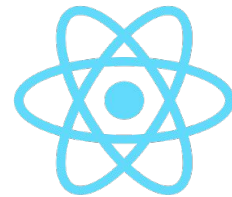
componentDidMount() example Code:
[componentDidMountExample.js](#)

static getDerivedStateFromProps()



static `getDerivedStateFromProps()`

- Invoked right before calling the render method, both on the initial mount and on subsequent updates
- It should return an object to update the state, or null to update nothing.
- This method exists for rare use cases where the state depends on changes in props over time

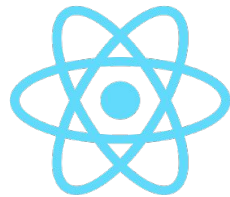


static `getDerivedStateFromProps()`

`getDerivedStateFromProps()` example Code:
[getDerivedStateFromPropsExample.js](#)

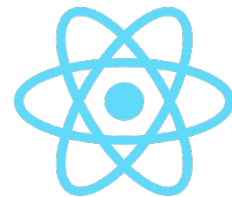
shouldComponentUpdate()





shouldComponentUpdate()

- It returns true by default.
- Whenever a component's state or props (its parent's state) is updated, the component re-renders we can stop updating the state if we want
- This method is not called for the initial render or when `forceUpdate()` is used
- `shouldComponentUpdate()` allows us to say: only update if the props you care about change

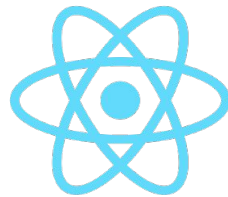


shouldComponentUpdate()

shouldComponentUpdate() example Code:
[shouldComponentUpdateExample.js](#)

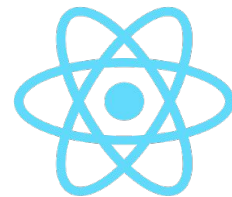
componentDidUpdate()





componentDidUpdate()

- It is called right after the component update takes place, except the first time when the component is rendered.
- `componentDidUpdate()` takes two arguments as parameters, `prevProps` and `prevState`
- This method is useful if you want to take any actions on DOM after received updated props from parent or re-render caused by `setState`
- This is also a suitable place when we want to log the new changes

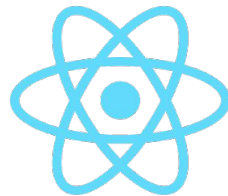


componentDidUpdate()

componentDidUpdate() example Code:
[componentDidUpdateExample.js](#)

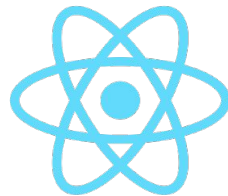
componentWillUnmount()





componentWillUnmount()

- This event (method) is invoked immediately before a component is being removed from the DOM
- Recommended to perform any necessary cleanup in this method, such as
 - Invalidating timers,
 - Canceling network requests, or
 - cleaning up any subscriptions that were created in `componentDidMount()`



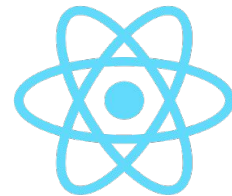
componentWillUnmount()

- `setState()` will not effect if called in `componentWillUnmount()` because the component cannot re-rendered once unmounted

componentWillUnmount() example:

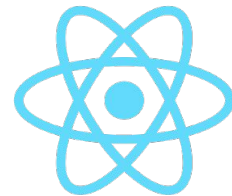
```
componentWillUnmount() {  
  window.removeEventListener('resize', this.resizeListener)  
}
```

Components file/directory structure



Components file/directory structure

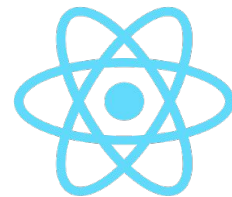
- Uptil now we have made a single App.js file and made all our component into it
- This is not at all a good approach
- Components should always made in different folders and .js files
- Keep app supporting function into utils folder
- Keep all images to one static resources folder etc



Components file/directory structure

Component splitted to file/folders example Code:
[SampleFolderStructure/](#)

Assignment # 2



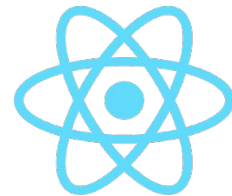
Assignment # 2

Create a todo application that

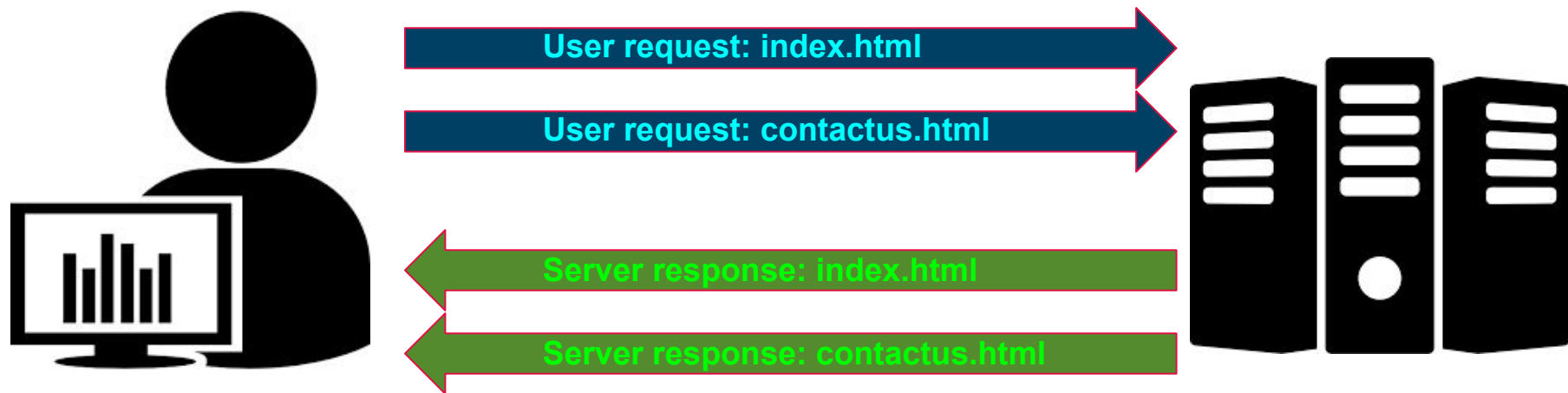
1. Fetch the todo records just once from <https://jsonplaceholder.typicode.com/>
2. Store them in local component state
3. Add a new todo functionality
4. Delete any todo functionality
5. List all todo functionality
6. Search any todo on Title field

Please note: Call API only once to get the records and then use/modify it locally using state

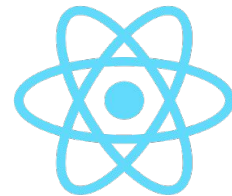
React Router



React Router

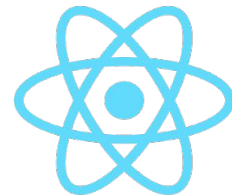


Normal flow of user request on browser and response from server



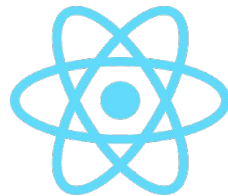
React Router

- Single-page applications can work in different ways
- In react we have only index.html
- React routers that user move to different pages in your web app without actually changing any page, even URL will show change of page
- One way is you can do have conditional rendering and move to different screens (components) on the browser
- Above way is not easy to manage and does not give browser back button



React Router

- Routing in a Single Page Application is the way to introduce some features to navigating the app through links, which are expected in normal web applications like:
 - The browser should change the URL when you navigate to a different screen
 - Deep linking should work: if you point the browser to a URL, the application should reconstruct the same view that was presented when the URL was generated
 - The browser back (and forward) button should work like expected

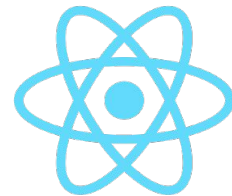


React Router v6

- React-router-dom is a library for routing in React SPA
- It provides all the previously discussed navigation features to SPA

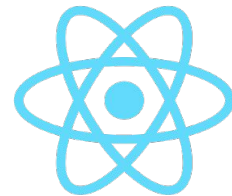
To install react-router-dom

```
aamir@ap-linux:~$ npm install history react-router-dom@next
```



React Router - `<BrowserRouter>`

- React-router-dom library provides `<BrowserRouter />` component
- The purpose of this component is to listen to URL changes
- When URL changes this component will make sure the correct screen (page) of your web app get render on the screen
- For React Router to work properly, we must have to wrap our whole app into a `BrowserRouter` component



<BrowserRouter> - index.js

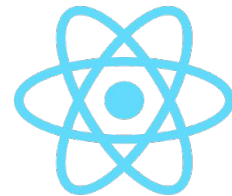
```
import React from 'react'
import ReactDOM from 'react-dom'
import './index.css'
import App from './Components/App'
import * as serviceWorker from './serviceWorker'

import { BrowserRouter } from 'react-router-dom'

ReactDOM.render(
  <BrowserRouter>
    <App />
  </BrowserRouter>,
  document.getElementById('root')
)
```

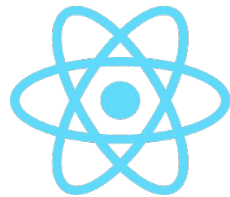
<Route>





<Route>

- Our routes will be defined in <Route> element
- We specify which path they will match and what component that should respond
- Let's say we want
 - **www.ourDomain.com/** to go to City list page
`<Route path="/" element={<CityList />} />`
 - **www.ourDomain.com/aboutus** to go to About us page
`<Route path="/aboutus" element={<AboutUs />} />`
- Before v6 we instead of element we had to use component attribute



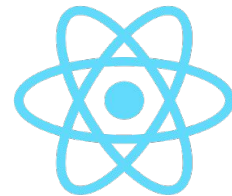
Nested routes <Route>

V5

```
<Route path='/cityList' element={<CityList />} />
<Route path='/cityList/mycity' element={<CityList />} />
<Route path='/cityList/:city' element={<CityList />} />
```

V6

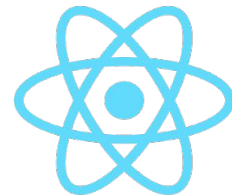
```
<Route path='cityList' element={<CityList/>} />
  <Route path='mycity' element={<MyCityList/>} />
  <Route path=':city' element={<OtherCityList />} />
</Route>
```

Passing props to routes <Route>

- For components where we pass props, before v6 element attribute that was known as component attribute had to be replaced by `render=()` => (`<your component>`) but thanks to v6 now it is simpler than ever.

```
<Route path='/cityList' element={<CityList city={['Karachi']}>} />
```

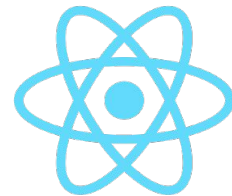


React Router

React Router example Code:
`RoutesExample/`

<Routes> in v6

<switch> in v5

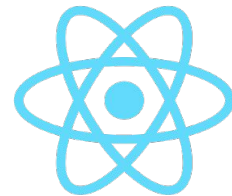


<Routes> Or <switch>

- We can add something like following to our code

```
<Route element={ <AnyComponent /> } />
```

- Only problem with this is it will render with every route no matter what path is provided
- To solve this problem we must wrap all above routes with <Routes></Routes>
- By doing this we actually are instructing that as URL path matches with path of any route first render that component and then jump out of switch block and do not go and check next routes



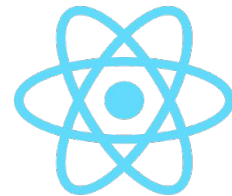
React Router

```
import { Routes } from 'react-router-dom'

<Routes>
  <Route path="/" element={<RandomNumber/>} />
  <Route path="/helloworld" element={<Helloworld/>} />
</Routes>
```

React Router example Code:
RoutesExample/

Programmatic navigation

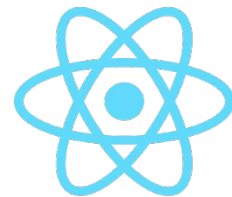


Programmatic navigation

- To programmatically navigate

```
import { useNavigate } from 'react-router-dom';
```

```
function MyButton() {  
  let navigate = useNavigate();  
  function handleClick() {  
    navigate('/home');  
  };  
  return <button onClick={handleClick}>Submit</button>;  
};
```



React Router

`<Link />`

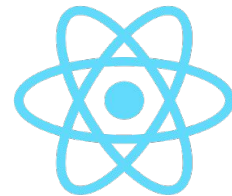
- It is a component of react-router-dom library

`<Link to="/helloworld">Click here</Link>`

same as

`<a href="/helloworld">Click here</a>`

- The purpose of this component is add provided route to URL
- Based on changes in URL `<BrowserRouter>` will sense the change in URL and render new UI
- Above all Back / Forward button of browser will also work

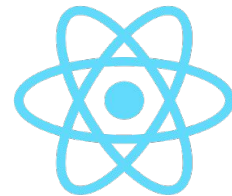


React Router

```
import React, { Component } from 'react';
import { Link } from 'react-router-dom'

class App extends Component {
  render() {
    return(
      <div>
        <Link to="/helloworld">Goto to Hello World Page</Link>
      </div>
    )
  }
  export default App
```

Assignment # 3



Assignment # 3

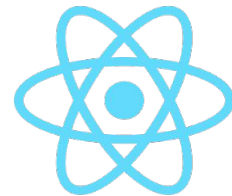
- Create a news website having separate routes
 - Top Headline news
 - Specific Country news
 - Sports news
 - Technology news
- For free news API choose any provider of your choice or visit newsapi.org

React Hooks

Enables State management and life cycle events
in function component

useState

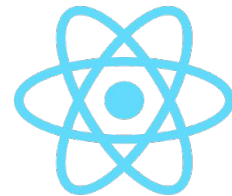
To manage state and state handler function



useState hook

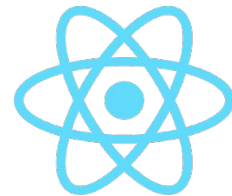
```
import React, { useState } from 'react';

function Example() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
    </div>
  )
}
```



useState hook

```
function ExampleWithManyStates() {  
  // Declare multiple state variables!  
  const [age, setAge] = useState(42);  
  const [fruit, setFruit] = useState('banana');  
  const [todos, setTodos] = useState([{ text: 'Learn Hooks' }]);  
  // ...  
}
```

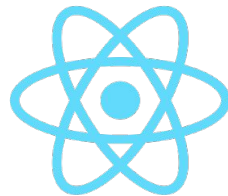


useState hook

```
function ExampleWithManyStates() {  
  // Declare multiple state variables!  
  const [age, setAge] = useState(42);  
  const [fruit, setFruit] = useState('banana');  
  const [todos, setTodos] = useState([{ text: 'Learn Hooks' }]);  
  // ...  
}
```


useEffect

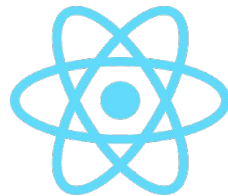
To manage side effect - serves the purpose of `componentDidMount` and `componentDidUpdate`



useEffect

```
import React, { useState, useEffect } from 'react';

function Example() {
  const [count, setCount] = useState(0);
  // Similar to componentDidMount and componentDidUpdate:
  useEffect(() => {
    if(count === 0){
      setCount(100) // sets initial count to 100
    }
  }, []); // << super important array, check the next slide
  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>Click me </button>
    </div>
  )
}
```

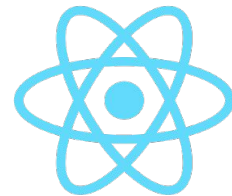


useEffect

```
useEffect(() => {  
  if(count === 0){  
    setCount(100) // sets initial count to 100  
  }  
}, []); // << When the array is empty, the effect runs only once.
```

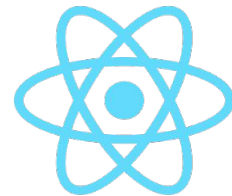
```
useEffect(() => {  
  if(count === 0){  
    setCount(100) // sets initial count to 100  
  }  
}, [count]); // << this useEffect will run only when there is a change in state of  
count variable
```

Context API

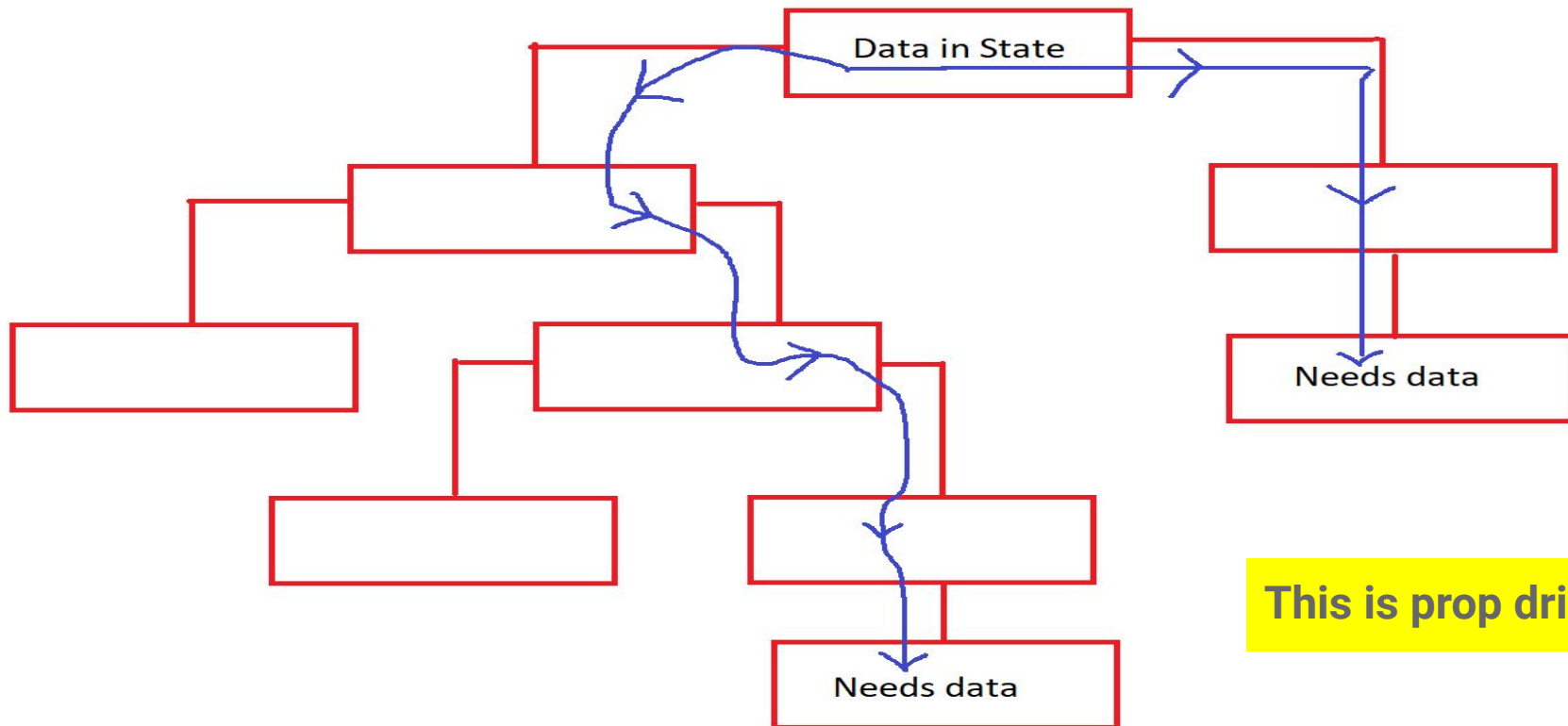


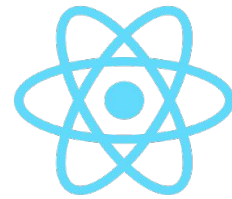
Context API

- Context API solves a major problem of **prop drilling** in react
- Prop drilling is actually passing prop from any component to deep nested component
- That we can only do by passing prop to every component that comes in a way
- Using Context API any component can access prop directly without getting it through hierarchical components

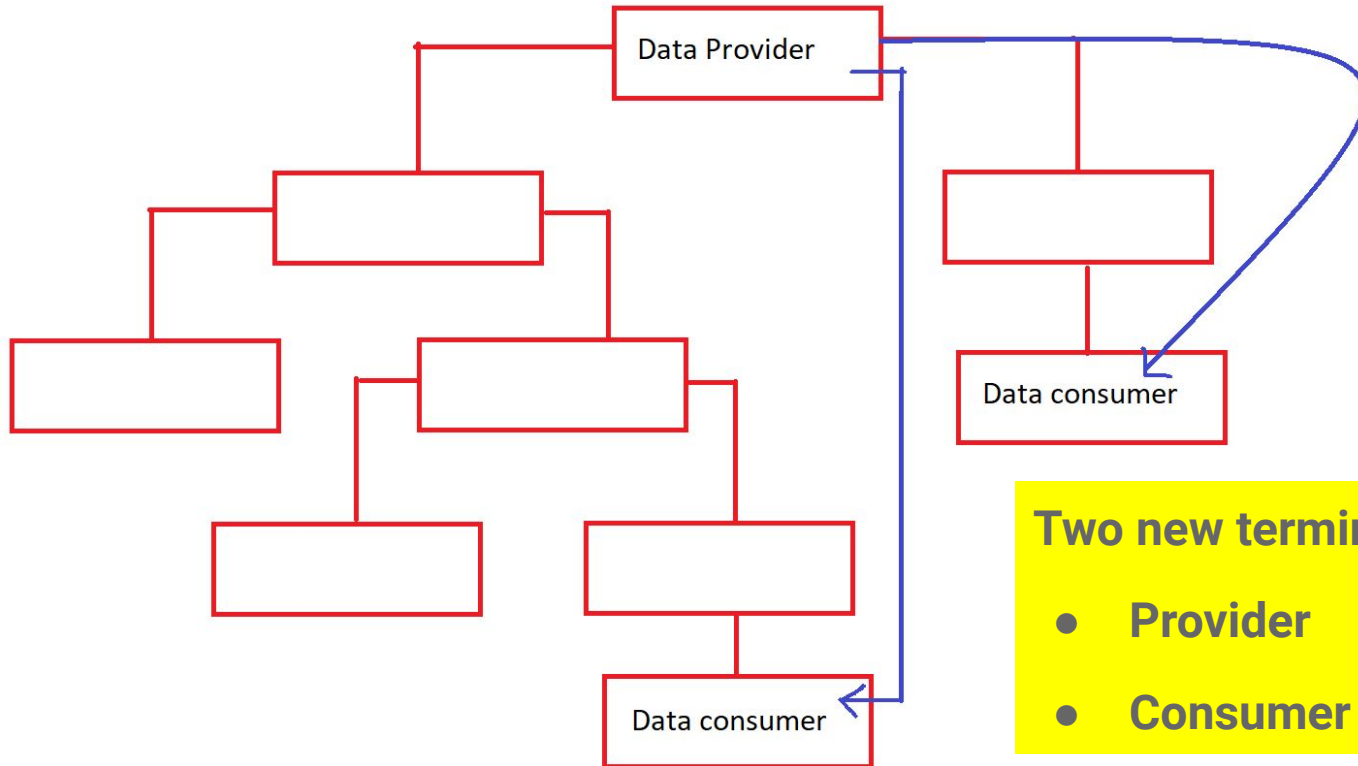


Problem with prop passing



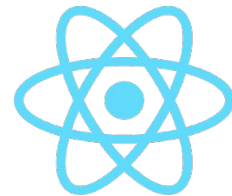


Context API - Solution



Two new terminologies

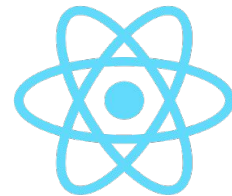
- **Provider**
- **Consumer**



Context API - Steps

1. Create the context
2. Create Provider
3. Wrap Application with Provider
4. Consume it

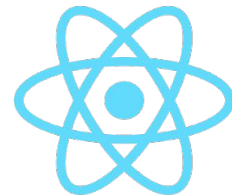
Let's do some practical



Initialize the Context

```
import React from 'react';  
  
// this is the equivalent to the createStore method of Redux  
// https://redux.js.org/api/createstore  
  
const MyContext = React.createContext();  
  
export default MyContext;
```

MyContext.js

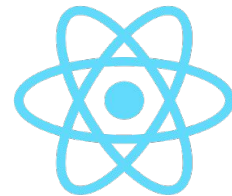


Create the Provider

```
class MyProvider extends Component {  
  state = { counter: 0 };  
  
  incrementCounter =  
this.incrementCounter.bind(this);  
  
  incrementCounter() {  
    const { counter } = this.state  
    this.setState({  
      counter: counter + 1  
    })  
  }  
}
```

```
render() {  
  return (  
    <MyContext.Provider  
      value={{  
        counter: this.state.counter,  
        incrementCounter:  
this.incrementCounter  
      }}  
    >  
      {this.props.children}  
    </MyContext.Provider>  
  );  
}
```

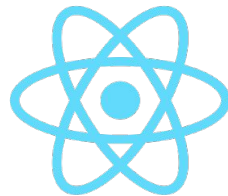
MyProvider.js



Wrap application with Provider

```
class App extends Component {  
  render() {  
    return (  
      <MyProvider>  
        <div className="App">  
          <CounterComponent />  
        </div>  
      </MyProvider>  
    );  
  }  
}
```

App.js



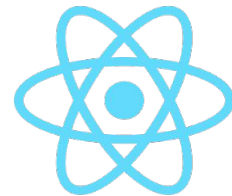
Consume the data from context

```
const CounterComponent = () => (  
  <MyContext.Consumer>  
    {context => (  
      <Fragment>  
        <h4>Counter:</h4> {context.counter}  
        <input type="button" onclick={context.incrementCounter}> + </button>  
        <input type="button" onclick={context.decrementCounter}> - </button>  
      </Fragment>  
    )}  
  </MyContext.Consumer>  
);
```

CounterComponent.js

useContext

Replaces `Context.Consumer`



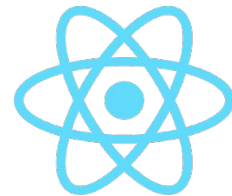
useContext

```
const CounterComponent = () => {  
  const context = useContext(MyContext);  
  return(  
    <Fragment>  
      <h4>Counter:</h4> {context.counter}  
      <input type="button" onclick={context.incrementCounter}> + </button>  
      <input type="button" onclick={context.decrementCounter}> - </button>  
    </Fragment>  
  )  
}
```

CounterComponent.js

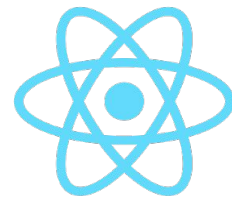
useReducer

Simplified version of Redux

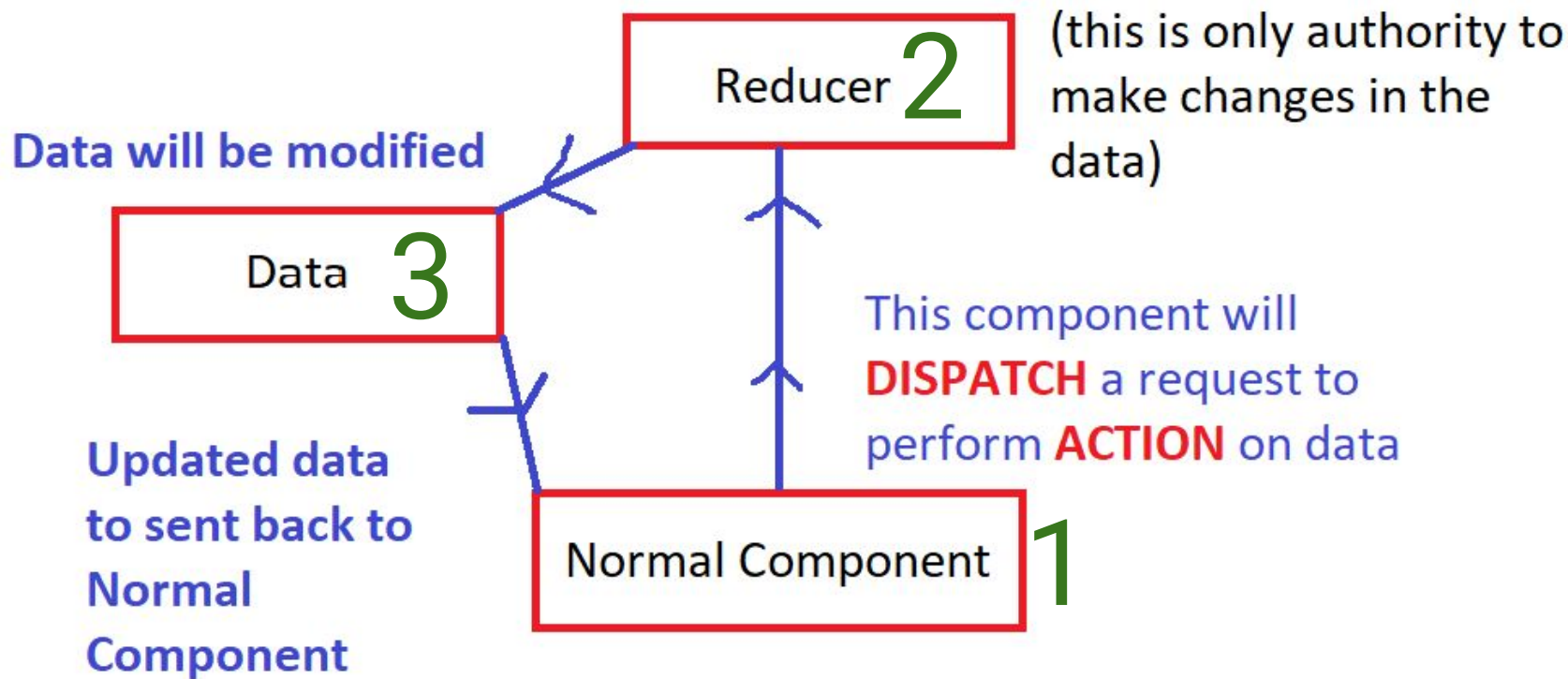


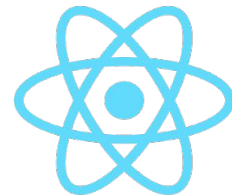
useReducer

- There will be a place for Data
- There will be a place that will responsible to change the data
- Component that will dispatch the request to perform action on the data



useReducer

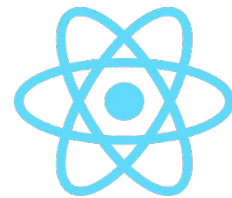




useReducer - reducer

```
export default function reducer(state, action) {  
  switch (action.type) {  
    case 'increment':  
      return {count: state.count + 1};  
    case 'decrement':  
      return {count: state.count - 1};  
    default:  
      throw new Error();  
  }  
}
```

reducer.js



useReducer - Counter with useReducer()

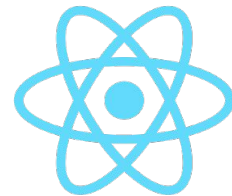
```
import reducer from '../reducer/reducer'

export default function Counter() {
  const initialState = {count: 0};

  const [state, dispatch] = useReducer(reducer, initialState);

  return (
    <>
      Count: {state.count}
      <button onClick={() => dispatch({type: 'decrement'})}>-</button>
      <button onClick={() => dispatch({type: 'increment'})}>+</button>
    </>
  );
}
```

Counter.js

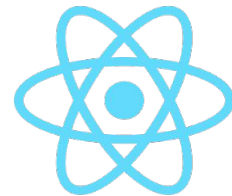


useReducer - App Component

```
class App extends Component {  
  render() {  
    return (  
      <div className="App">  
        <CounterComponent />  
      </div>  
    );  
  }  
}
```

App.js

Assignment # 4

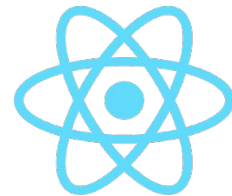


Assignment # 4

Create a Weather application that

1. Fetch the records from any free weather api
2. Store them in local component by using context API
3. Search weather for any counter or city
4. Also provide the default weather on initial load based on user current location (user current location can be found by the user current IP address)

Assignment # 5



Assignment # 5

Create a dictionary application that

1. Fetch the records from any free dictionary api
2. Store them in local component useReducer
3. Search word to get the meaning
4. Maintain previously searched 10 words and there meaning in local storage.

Thank you

